

SME Financial Intelligence System

Implementation Report Inspired by the MiroFish Workflow

A practical blueprint for turning messy SME data into structured, validated, traceable company memory and financial statements.

Prepared: 11 May 2026

Core Thesis

The product should not be framed as “AI generates financial statements.” The serious version is: messy SME records -> structured company memory -> validated financial statements -> management and audit interaction.

1. Executive Summary

This report defines a practical implementation plan for an AI-assisted financial intelligence system for SMEs. The system is inspired by the MiroFish workflow, but it adapts the logic from social simulation into financial data structuring, accounting validation, and evidence-backed reporting.

Flexible SME raw data input

- > file classification and schema mapping
- > business record extraction
- > business ontology construction
- > company knowledge graph construction
- > process validation and importance scoring
- > accounting mapping and journal generation
- > financial statement generation
- > management report and deep interaction chat

The reason this matters: SMEs do not store data in one clean format. They use ERP exports, Excel files, POS reports, supplier invoices, receipts, inventory records, branch reports, and bank statements. A normal accounting automation tool expects clean input; this project should convert messy input into standardized business memory.

2. Problem Background

- SME financial data is often fragmented across sales, inventory, purchasing, banking, and accounting files.
- Different companies use different column names, file structures, languages, and manual habits.
- A financial statement number must be traceable to source records, not guessed by a language model.
- The biggest technical challenge is not report writing; it is converting messy operational data into structured, validated accounting memory.

Weak Approach to Avoid

Upload messy files -> ask an LLM to understand everything -> generate a financial statement. This is risky because the model may hallucinate, misclassify columns, miss missing records, or invent explanations.

3. How the MiroFish Workflow Is Adapted

MiroFish stage	Original purpose	Financial AI adaptation
Ontology Generation	Define social actors and relationships	Define business objects, records, accounts, departments, and relationships
Graph Construction	Build memory from seed documents	Build company knowledge graph from ERP, sales, inventory, purchase, and bank records
Parallel Simulation	Run social agents in a simulated environment	Run process validation agents for accounting controls and exception detection
Report Generation	Generate prediction or scenario report	Generate financial statements, exception reports, and management explanations
Deep Interaction	Chat with the generated	Ask management/audit questions over

	simulation/report world	evidence-backed company memory
--	-------------------------	--------------------------------

Important correction: the social simulation layer should not be copied directly. For financial reporting, the replacement should be process validation, accounting rules, audit trail, and human review.

4. Complete Working Sequence

1. User creates a company/project and selects a reporting period.
2. User uploads raw files such as Excel, CSV, ERP exports, invoices, receipts, bank statements, and inventory files.
3. Backend saves raw files and file metadata.
4. File parser reads the data into tables or extracted text.
5. File classifier detects the business meaning: sales file, supplier invoice file, inventory file, bank statement, chart of accounts, etc.
6. Schema mapper maps messy columns to standard fields such as branch_id, sku_id, sales_amount, cost_amount, invoice_id, supplier_id, and payment_amount.
7. Record extractor converts rows or documents into standardized business records.
8. Entity resolver identifies branches, suppliers, SKUs, accounts, documents, and departments, merging duplicates where needed.
9. Business ontology defines the allowed objects and relationships.
10. Graph builder links records and entities into a company knowledge graph.
11. Validation engine checks process and accounting logic.
12. Importance scorer ranks records, relationships, files, accounts, branches, and statement lines.
13. Accounting mapper generates accounting impacts and journal entries.
14. Statement generator creates trial balance, income statement, balance sheet, and exception reports.
15. Report generator explains results with evidence.
16. Chat agent answers management and audit questions using graph, validation, scores, journal entries, and evidence.

5. Data Architecture

The recommended database architecture is hybrid: relational tables for accounting reliability, JSON fields for flexible SME data, and graph-style relationship tables for business connections.

Relational database
 + flexible JSON fields
 + graph-style relationships
 + file storage
 + optional vector/LLM memory layer

Layer	Main tables	Purpose
Company and period	companies, periods	Define company identity and reporting period
Raw file	files, file_sheets	Store uploaded file metadata and sheet information
Schema mapping	column_mappings	Map raw columns to standard business

		fields
Parsed records	raw_records, business_records	Preserve original rows and standardized objects
Business entities	entities	Store graph nodes such as Branch, SKU, Supplier, Invoice, Account
Graph relationships	relationships	Connect business objects and records
Validation	validation_results	Store passed, warning, failed, and needs-review checks
Importance	importance_scores	Rank materiality, exceptions, risks, and statement relevance
Accounting	accounts, journal_entries, journal_lines	Generate accounting impacts and debit/credit records
Statements	financial_statements, statement_lines, statement_line_evidence	Generate statement lines with evidence
Reports/chat	reports, chat_sessions, chat_messages	Store explanations and evidence-backed Q&A

6. Database Structure Summary

Table	Key fields	Reason it exists
companies	company_id, company_name, industry, currency	Separates each SME and its context
periods	period_id, start_date, end_date, status	Every statement belongs to a period
files	file_id, path, file_type, business_process, confidence	Tracks uploaded files and their detected meaning
column_mappings	raw_column_name, standard_field_name, confidence	Solves inconsistent SME column names
raw_records	raw_data_json, row_number, file_id	Preserves original evidence
business_records	record_type, normalized_data_json, amount, status	Creates standardized business objects
entities	entity_type, entity_name, external_code, attributes_json	Stores graph nodes
relationships	source_entity_id, relationship_type, target_entity_id, evidence_json	Stores graph edges
validation_results	validation_type, status, severity, message	Stores control checks
importance_scores	materiality, exception, risk, total_score, level	Prioritizes records and risks
accounts	account_code, account_name, account_type, statement_type	Stores chart of accounts
journal_entries / lines	entry_date, debit, credit, account_id	Supports proper statement generation
statement_line_evidence	statement_line_id, source_record_id, file_id	Traces report numbers back to evidence

7. Data Construction Process

7.1. File Parsing

- Read Excel and CSV in the MVP using pandas and openpyxl.
- Later add PDF, invoice, receipt, and bank-statement OCR.
- Store raw files and raw rows before any transformation for audit trail.

7.2. File Classification

- Classify file types such as sales_transaction_file, supplier_invoice_file, inventory_movement_file, bank_statement_file, chart_of_accounts_file, and waste_record_file.
- Use rules first: column names, sheet names, file names, and sample values.
- Use LLM only when the pattern is unclear.

7.3. Schema Mapping

- Map messy columns to standard fields.
- Show mapping confidence and let humans approve or edit mapping.
- Store mapping templates per company to improve future uploads.

Messy SME column	Standard field	Meaning
Sales / Revenue / TotalSale	sales_amount	Revenue amount for sales transaction
ItemCode / SKU / ProductCode	sku_id	Product or stock keeping unit
Branch / Store / Location	branch_id	Branch or store where transaction happened
Cost / UnitCost / COGS	cost_amount	Cost amount related to sale or inventory
Supplier / Vendor	supplier_id	Supplier identity
InvoiceNo / BillNo	invoice_id	Supplier or customer invoice identifier

7.4. Record Extraction and Entity Resolution

- Convert each row into a business record such as SalesTransaction, SupplierInvoice, InventoryMovement, Payment, or WasteRecord.
- Resolve duplicate entities such as supplier names written differently across files.
- Link each record back to raw file, sheet, and row number.

8. Backend Architecture

```

backend/
  main.py
  config.py
  api/
    upload.py
    classify.py
    schema.py
    graph.py
    validation.py
    statements.py
    report.py
    chat.py
  services/
    file_parser.py
    file_classifier.py
    schema_mapper.py
    record_extractor.py
    ontology_service.py
    entity_resolver.py
    graph_builder.py
    validation_engine.py
    importance_scorer.py

```

```

accounting_mapper.py
statement_generator.py
report_generator.py
chat_agent.py
agents/
sales_agent.py
purchasing_agent.py
inventory_agent.py
bank_agent.py
tax_agent.py
audit_agent.py

```

Component	Responsibility
main.py	Starts the backend app and registers API routes
config.py	Stores database URL, upload folder, LLM API key, model name, file limits, graph settings
api/	Receives frontend requests such as upload, classify, graph build, validation, report, and chat
services/	Performs the actual processing work
agents/	Runs process-specific validation and accounting checks
models/	Defines business objects, validation outputs, and statement structures

9. API Design

Endpoint	Purpose	Output
POST /api/upload	Upload raw SME files	file_id, file metadata
POST /api/classify/{file_id}	Detect business file type	file_type, confidence
POST /api/schema/map/{file_id}	Suggest standard column mapping	column mappings, confidence
POST /api/records/extract/{file_id}	Create standardized business records	record count, extraction status
POST /api/graph/build	Build company knowledge graph	nodes, edges, graph status
POST /api/validation/run	Run validation rules	passed/warning/failed results
POST /api/statements/generate	Generate statements from journal lines	statement_id and line items
POST /api/report/generate	Generate management and exception report	report_id, report output
POST /api/chat	Ask questions over company memory	answer with evidence

10. Frontend Architecture

The frontend is the control panel. It should not contain core accounting logic. For MVP, use Streamlit or a simple web interface. For a serious product, use Vue or React.

Page	Purpose
Company setup	Create company and reporting period
File upload	Upload Excel/CSV first, later PDF/image/bank files
Classification result	Show detected file type and confidence
Schema mapping review	Approve or edit raw-column to standard-field mapping
Validation dashboard	Show passed, warning, failed, and critical exceptions
Importance dashboard	Show high-risk records, branches, accounts, suppliers, and files

Financial statement viewer	Show income statement, balance sheet, trial balance, and evidence
Management chat	Ask evidence-backed questions about the company data

11. Knowledge Graph Design

The knowledge graph is the company memory backbone. It connects records that would normally sit in separate files.

Graph node type	Examples
Operational entities	Company, Branch, Department, Supplier, Customer, SKU, ProductCategory
Transaction entities	SalesTransaction, SupplierInvoice, PurchaseOrder, GoodsReceipt, Payment, BankTransaction
Inventory entities	InventoryMovement, WasteRecord, StockAdjustment
Accounting entities	Account, JournalEntry, StatementLine
Evidence entities	Document, File, RawRecord

Relationship	Meaning
BRANCH_SOLD_SKU	A branch sold a product
SUPPLIER_ISSUED_INVOICE	A supplier issued an invoice
INVOICE_MATCHES_PO	Invoice matches purchase order
INVOICE_MATCHES_GRN	Invoice matches goods received note
PAYMENT_SETTLES_INVOICE	Payment settles invoice
TRANSACTION_POSTS_TO_ACCOUNT	Transaction affects accounting account
MOVEMENT_AFFECTS_INVENTORY	Inventory movement changes stock balance
STATEMENT_LINE_SUPPORTED_BY_RECORD	Statement line has source evidence

12. Simulation and Process-Agent Layer

For this financial project, “simulation” should not mean fake social-media roleplay. It should mean process simulation and validation: checking whether business records logically flow through sales, purchasing, inventory, banking, tax, and reporting processes.

Agent	Checks
Sales / Order-to-Cash Agent	Sales transaction completeness, branch/SKU existence, revenue mapping, discount abnormality, COGS link
Procure-to-Pay Agent	Supplier invoice, purchase order match, goods receipt match, duplicate invoices, AP impact
Inventory-to-COGS Agent	Stock movements, negative stock, waste records, COGS support, branch transfers
Bank Reconciliation Agent	Payments, deposits, unmatched bank transactions, cash/bank impact
Tax Agent	VAT/tax fields, tax-sensitive records, missing tax classification
Record-to-Report Agent	Journal entry creation, trial balance, statement generation
Internal Audit Agent	Missing evidence, high-risk exceptions, manual journal entries, duplicate or abnormal transactions

Design Principle

Rules for correctness. Graph for relationships. LLM for ambiguity and explanation. Human review for high-risk uncertainty.

13. LLM Role and Guardrails

Good LLM use	Bad LLM use
Classify unclear file types	Calculate final financial statement numbers alone
Suggest schema mappings from messy columns	Validate debit equals credit alone
Extend business ontology when new data types appear	Invent missing transactions
Explain exceptions and management insights	Approve high-risk records without evidence
Answer chat questions using retrieved evidence	Replace accounting rules or accountant review

The LLM should receive controlled context: column names, sample rows, summaries, graph facts, validation results, exceptions, and evidence. Do not send full ERP files directly to the LLM unless absolutely necessary.

14. Validation, Importance Scoring, and Evidence

14.1. Validation Engine

- Three-way match: invoice, purchase order, and goods receipt.
- Duplicate invoice and duplicate payment checks.
- Sales records must have branch, SKU, quantity, amount, and account mapping.
- Inventory movement should not create unexplained negative stock.
- Journal entries must balance debit and credit.
- High-risk exceptions should be blocked or sent to human review.

14.2. Importance Score

Importance Score =
 materiality score
 + exception score
 + risk score
 + account criticality score
 + relationship score

Score type	Meaning
Materiality	How large the amount is relative to account, branch, supplier, or period
Exception	Whether validation failed or key evidence is missing
Risk	Fraud/error possibility such as duplicate payment or new supplier
Account criticality	Whether it affects cash, revenue, inventory, AP, AR, tax, or COGS
Relationship	Whether key business links are complete or broken

15. Financial Statement Generation

Statements should be generated from mapped accounting records and journal entries, not from free-form LLM text.

Business event	Accounting impact
Cash sale	Dr Cash / Cr Sales Revenue; Dr COGS / Cr Inventory
Credit sale	Dr Accounts Receivable / Cr Sales Revenue; Dr COGS / Cr Inventory
Supplier invoice for inventory	Dr Inventory / Cr Accounts Payable
Payment to supplier	Dr Accounts Payable / Cr Cash
Inventory write-off or waste	Dr Waste/COGS Expense / Cr Inventory
Bank fee	Dr Bank Charges Expense / Cr Cash

- MVP should start with revenue, COGS, gross profit, inventory exceptions, supplier invoice exceptions, and evidence links.
- Later product should add full trial balance, balance sheet, cash flow statement, tax support, and closing workflow.
- Every statement line must link back to source records and files.

16. MVP vs Actual Product

Area	MVP	Actual product
Input	Excel and CSV only	Excel, CSV, PDF, receipt images, bank statements, ERP connectors
Database	SQLite	PostgreSQL with backup and access control
Graph	SQL relationships table or NetworkX	Neo4j, Zep-style temporal graph, or production graph layer
Frontend	Streamlit/simple UI	Vue or React dashboard
LLM	One cheap/reliable LLM API	Multi-model routing and cost controls
OCR	Skip at first	Google/Azure/AWS OCR or document intelligence
Accounting output	Income statement basics and exceptions	Trial balance, income statement, balance sheet, cash flow, tax reports
Controls	Basic validation rules	Human review workflow, audit trail, permissions, versioning
Users	One company/one period	Multi-company, multi-user, role-based access

16.1. MVP Build Order

17. Build file upload and metadata storage.
18. Build Excel/CSV parser.
19. Build file classification for sales, purchase, inventory, and chart of accounts.
20. Build schema mapping with review screen.
21. Extract standardized business records.
22. Create entities and relationships.
23. Run validation rules.
24. Calculate importance scores.
25. Generate basic income statement and exception report.

26. Add simple management chat over validated evidence.

17. Important Missing Pieces to Make the System Reasonable

Missing piece	Why it matters
Human-in-the-loop review	High-risk financial decisions cannot be fully automated safely
Audit trail	Every statement number must trace back to source evidence
Data quality score	Importance says what matters; quality says whether the data is usable
Confidence score	Needed for mapping, extraction, entity resolution, and LLM explanations
Versioning	Files, mappings, validations, and statements will change over time
Master data management	Branch, SKU, supplier, account, and department data must be cleaned
Security and permissions	Financial data needs role-based access, encryption, API key protection, and logs
Error handling	The system must fail safely when files are unreadable or mappings are uncertain
Evaluation metrics	Need proof: mapping accuracy, extraction accuracy, statement difference, review time saved
Accounting rule library	Rules for sales, purchases, inventory, payments, waste, tax, and journal entries

18. Implementation Roadmap

Phase	Goal	Deliverables
Phase 1: Prototype	Prove messy Excel/CSV can become structured records	Upload, parse, classify, map, extract, simple graph, basic validation
Phase 2: MVP	Make it usable for one SME and one period	Mapping review UI, validation dashboard, importance scoring, journal entries, basic statements
Phase 3: Pilot	Test with real SME data	OCR, bank import, entity resolution, exception workflow, accountant comparison
Phase 4: Product	Prepare for real users	Cloud deployment, PostgreSQL, graph database, user permissions, audit logs, production frontend

19. Key Risks and Mitigations

Risk	Impact	Mitigation
Wrong schema mapping	Downstream records and statements become wrong	Mapping confidence, sample preview, human approval, templates
LLM hallucination	False explanations or classifications	Controlled context, evidence requirement, rule-based validation
Weak accounting logic	Unreliable statements	Deterministic rule library and accountant comparison
Messy master data	Duplicate suppliers/SKUs/branches fragment graph	Entity resolution and manual merge tools
Overbuilding too early	Project becomes too complex and dies	Start with Excel/CSV, income statement

		basics, and evidence trail
Security failure	Sensitive financial data exposure	Authentication, permissions, encryption, API key management, logs

20. Reference Concepts

The implementation logic is inspired by these concepts and technologies:

- MiroFish workflow pattern: ontology generation, graph construction, simulation, report generation, and deep interaction.
- Knowledge graph design: entities, relationships, source evidence, and retrieval over connected business facts.
- FastAPI-style backend architecture: API routes, service modules, and structured application configuration.
- Zep/MiroFish-style memory: graph-based memory and downstream retrieval, adapted here into company memory rather than social simulation memory.
- Accounting control principles: source documents, validation, journal entries, trial balance, financial statements, and audit trail.

21. Final Summary

This project is feasible if it is built as a structured data-to-memory-to-report pipeline. The MiroFish-style workflow is valuable because it shows how flexible raw input can be transformed through ontology, graph construction, downstream reasoning, report generation, and deep interaction. However, the financial system must replace social simulation with accounting validation, process agents, importance scoring, audit trail, and human review.

Final Product Statement

An SME financial intelligence platform that accepts flexible raw business data, maps it into a standard business ontology, constructs a company knowledge graph, validates records through process-specific controls, scores risk and importance, generates traceable financial statements, and supports management/audit interaction through evidence-backed chat.